

Sau khi Chương 2 đã khảo sát nhu cầu và xác định các yêu cầu chính của hệ thống, chương này trình bày các nền tảng lý thuyết và công nghệ được sử dụng để xây dựng trợ lý ảo giáo dục. Trọng tâm của chương không phải là mô tả chi tiết mã nguồn, mà là làm rõ các khái niệm cần thiết để hiểu các quyết định thiết kế ở các chương sau. Nội dung được tổ chức từ nền tảng xử lý ngôn ngữ tự nhiên, mô hình ngôn ngữ lớn, truy xuất tăng cường sinh, đến các cơ chế điều phối, đa tác tử và đánh giá chất lượng hệ thống.

0.1 Kiến thức nền tảng về Trí tuệ Nhân tạo và Xử lý ngôn ngữ tự nhiên

Hệ thống trợ lý ảo giáo dục xử lý đầu vào chủ yếu ở dạng ngôn ngữ tự nhiên. Vì vậy, các thành phần như tách đơn vị văn bản, biểu diễn ngữ nghĩa và mô hình hóa quan hệ giữa các từ đóng vai trò nền tảng cho cả pha truy xuất tri thức và pha sinh câu trả lời.

0.1.1 Tokenization

Tokenization là quá trình chuyển văn bản thô thành các đơn vị nhỏ hơn gọi là token. Token có thể là ký tự, âm tiết, từ hoặc một phần của từ tùy theo bộ mã hóa được sử dụng. Với các mô hình ngôn ngữ hiện đại, tokenization thường không chỉ dựa trên khoảng trắng, vì cách làm này khó xử lý từ mới, tên riêng hoặc các biến thể chính tả. Một kỹ thuật phổ biến là Byte-Pair Encoding (BPE), trong đó hệ thống bắt đầu từ các đơn vị ký tự rồi lặp lại quá trình ghép các cặp xuất hiện thường xuyên để tạo token mới.

Đối với tiếng Việt, tokenization cần được xem xét cẩn thận vì một từ có thể gồm nhiều âm tiết tách nhau bằng khoảng trắng. Ví dụ, cụm “học sinh” và “sinh học” đều gồm hai âm tiết nhưng mang nghĩa khác nhau. Nếu chỉ tách theo khoảng trắng, hệ thống có thể làm mất ranh giới từ ghép và ảnh hưởng đến biểu diễn ngữ nghĩa, đặc biệt khi xây dựng kho tri thức hoặc tối ưu truy xuất.

0.1.2 Biểu diễn vector

Mô hình máy học không xử lý trực tiếp văn bản như con người đọc câu chữ, mà cần biểu diễn văn bản thành vector số. Embedding là kỹ thuật ánh xạ token, câu hoặc đoạn văn thành vector dày đặc trong không gian nhiều chiều. Hai đoạn văn có ý nghĩa gần nhau thường có vector nằm gần nhau hơn so với các đoạn văn không liên quan.

Độ tương đồng giữa hai vector thường được tính bằng Cosine Similarity:

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \quad (1)$$

Trong đó, A và B là hai vector biểu diễn văn bản. Giá trị cosine càng gần 1 thì hai vector càng cùng hướng, tức hai văn bản càng gần nhau về mặt ngữ nghĩa. Đây là cơ sở cho tìm kiếm ngữ nghĩa trong hệ thống RAG.

0.1.3 Kiến trúc Transformer và cơ chế Attention

Transformer là kiến trúc nền tảng của nhiều mô hình ngôn ngữ hiện đại. Điểm khác biệt chính của Transformer so với các mô hình tuần tự trước đây là cơ chế tự chú ý, cho phép mô hình đánh giá mức độ liên quan giữa các token trong cùng một ngữ cảnh.

Cơ chế Scaled Dot-Product Attention được mô tả bởi công thức:

$$Attention(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (2)$$

Trong đó, Q là truy vấn, K là khóa, V là giá trị, và d_k là số chiều của vector khóa. Thành phần $\frac{QK^T}{\sqrt{d_k}}$ tính mức độ tương hợp giữa truy vấn và khóa, sau đó hàm softmax chuyển kết quả thành trọng số chú ý. Nhờ vậy, mô hình có thể ưu tiên các phần thông tin quan trọng trong câu hoặc đoạn văn.

0.2 Mô hình ngôn ngữ lớn

Mô hình ngôn ngữ lớn là thành phần sinh và suy luận ngôn ngữ trong hệ thống. Tuy nhiên, trong một trợ lý giáo dục, mô hình ngôn ngữ không nên được sử dụng như nguồn tri thức duy nhất. Các câu trả lời cần được neo vào dữ liệu sách giáo khoa và được kiểm soát bởi các lớp truy xuất, điều phối và đánh giá chất lượng.

0.2.1 Bản chất dự đoán token tiếp theo

LLM được huấn luyện để dự đoán token tiếp theo dựa trên chuỗi token đã có. Với một chuỗi có độ dài T , xác suất sinh chuỗi có thể được biểu diễn như sau:

$$P(w_1, w_2, \dots, w_T) = \prod_{t=1}^T P(w_t \mid w_1, \dots, w_{t-1}) \quad (3)$$

Trong đó, w_t là token tại vị trí t , còn $w_1, \dots, w_{t-1}$ là ngữ cảnh trước đó. Khi sinh văn bản, mô hình chọn token tiếp theo dựa trên phân phối xác suất này. Cơ chế đó giúp LLM tạo được câu trả lời tự nhiên, nhưng cũng khiến mô hình có thể sinh thông tin hợp lý về mặt ngôn ngữ mà không đúng với tài liệu nguồn.

0.2.2 Prompt và khả năng gọi công cụ

Prompt là chỉ thị đầu vào dùng để định hướng hành vi của mô hình. Trong hệ thống trợ lý giáo dục, prompt không chỉ chứa câu hỏi của người dùng mà còn có

thể chứa ngữ cảnh truy xuất từ sách giáo khoa, yêu cầu định dạng câu trả lời, hoặc ràng buộc về phạm vi lớp và bộ sách.

Khả năng gọi công cụ cho phép LLM không chỉ sinh văn bản tự do mà còn phát ra yêu cầu có cấu trúc để hệ thống thực hiện một thao tác cụ thể. Trong triển khai thực tế, việc gọi công cụ cần một lớp điều phối bên ngoài để kiểm soát quyền truy cập, kiểu dữ liệu vào ra và xử lý lỗi.

0.2.3 Hiện tượng hallucination

Hallucination là hiện tượng mô hình sinh ra thông tin không có cơ sở trong dữ liệu nguồn hoặc trái với thực tế. Nguyên nhân đến từ bản chất dự đoán xác suất của LLM, giới hạn của dữ liệu huấn luyện và việc mô hình không tự kiểm chứng được tri thức tại thời điểm trả lời. Với bài toán giáo dục, hiện tượng này có rủi ro cao vì câu trả lời sai có thể ảnh hưởng trực tiếp đến quá trình học tập.

Vì vậy, hệ thống sử dụng kiến trúc RAG để truy xuất ngữ cảnh từ sách giáo khoa trước khi sinh câu trả lời. Cách tiếp cận này giúp câu trả lời có cơ sở dữ liệu rõ ràng hơn so với việc chỉ dựa vào tri thức tham số của mô hình ngôn ngữ lớn.

0.3 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) là kiến trúc kết hợp truy xuất thông tin và sinh văn bản. Thay vì để LLM trả lời trực tiếp từ tri thức đã học, hệ thống trước hết tìm các đoạn tài liệu liên quan, sau đó đưa các đoạn này vào prompt để mô hình sinh câu trả lời dựa trên ngữ cảnh.

0.3.1 Kiến trúc RAG cơ bản

Một hệ thống RAG thường gồm hai pha. Ở pha chuẩn bị dữ liệu, tài liệu được làm sạch, chia thành các đoạn nhỏ, chuyển thành embedding và lưu cùng metadata. Ở pha truy vấn, câu hỏi của người dùng được phân tích, chuyển thành vector hoặc từ khóa tìm kiếm, sau đó hệ thống lấy các đoạn tài liệu phù hợp nhất để đưa vào LLM.

Trong đề tài này, dữ liệu truy xuất chính là nội dung sách giáo khoa Tin học THPT thuộc các bộ sách được hỗ trợ. Do dữ liệu nguồn có cấu trúc theo lớp, bộ sách, chủ đề, bài học và mục nhỏ, metadata của từng đoạn văn có ý nghĩa quan trọng. Metadata giúp hệ thống giới hạn phạm vi truy xuất và tránh trộn lẫn nội dung giữa các lớp hoặc bộ sách khi người dùng đã nêu rõ yêu cầu.

0.3.2 Phân mảnh dữ liệu

Chunking là quá trình chia tài liệu dài thành các đoạn nhỏ hơn để phù hợp với giới hạn ngữ cảnh của mô hình và để tăng độ chính xác truy xuất. Nếu đoạn quá dài, kết quả truy xuất có thể chứa nhiều thông tin thừa; nếu đoạn quá ngắn, ngữ

cảnh có thể bị đứt đoạn.

Các chiến lược chunking thường gặp gồm cắt theo kích thước cố định, cắt theo ranh giới đoạn hoặc câu, và cắt theo cấu trúc phân cấp của tài liệu. Với dữ liệu sách giáo khoa, hướng phân cấp có lợi thế vì giữ lại quan hệ giữa lớp, chương, bài và mục nội dung.

0.3.3 Tìm kiếm vector

Tìm kiếm vector sử dụng embedding để tìm các đoạn văn gần câu hỏi về mặt ngữ nghĩa. Cả câu hỏi và tài liệu đều được ánh xạ vào cùng một không gian vector. Sau đó, hệ thống dùng một độ đo tương đồng, chẳng hạn Cosine Similarity, để xếp hạng các đoạn tài liệu.

Cách tiếp cận này phù hợp với các câu hỏi diễn đạt khác với tài liệu gốc nhưng có cùng ý nghĩa. Tuy nhiên, tìm kiếm vector có thể kém hiệu quả với các từ khóa hiếm, tên lệnh, thuật ngữ kỹ thuật hoặc ký hiệu cụ thể. Vì vậy, hệ thống cần kết hợp thêm tìm kiếm từ khóa.

0.4 Truy xuất thông tin nâng cao

Trong một trợ lý giáo dục, chất lượng câu trả lời phụ thuộc mạnh vào chất lượng ngữ cảnh được truy xuất. Nếu hệ thống lấy sai hoặc thiếu tài liệu, LLM khó sinh câu trả lời đúng dù prompt được thiết kế tốt. Vì vậy, dự án sử dụng kết hợp tìm kiếm từ khóa, tìm kiếm ngữ nghĩa, hợp nhất xếp hạng và xếp hạng lại.

0.4.1 Tìm kiếm từ khóa

Tìm kiếm từ khóa dựa trên sự trùng khớp bề mặt giữa truy vấn và tài liệu. Một phương pháp nền tảng là TF-IDF, trong đó điểm của một từ tăng khi từ đó xuất hiện nhiều trong tài liệu hiện tại và giảm khi từ đó xuất hiện quá phổ biến trong toàn bộ tập tài liệu:

$$TF_IDF(t, d, D) = TF(t, d) \times IDF(t, D) \quad (4)$$

BM25 là một biến thể được sử dụng rộng rãi hơn trong truy xuất thông tin vì có cơ chế điều chỉnh ảnh hưởng của tần suất từ và độ dài tài liệu:

$$Score_{BM25}(Q, d) = \sum_{q_i \in Q} IDF(q_i) \cdot \frac{f(q_i, d) \cdot (k_1 + 1)}{f(q_i, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{avgdl}\right)} \quad (5)$$

Trong đó, Q là truy vấn, $f(q_i, d)$ là số lần từ khóa xuất hiện trong tài liệu d , $|d|$ là độ dài tài liệu và $avgdl$ là độ dài trung bình của các tài liệu. Các tham số k_1 và b điều chỉnh mức bão hòa tần suất và mức phạt theo độ dài tài liệu.

0.4.2 Tìm kiếm lai và hợp nhất xếp hạng

Tìm kiếm lai kết hợp ưu điểm của tìm kiếm từ khóa và tìm kiếm ngữ nghĩa. Tìm kiếm từ khóa phù hợp với thuật ngữ chính xác, còn tìm kiếm ngữ nghĩa phù hợp với cách diễn đạt linh hoạt. Khi hai cơ chế trả về hai danh sách kết quả khác nhau, hệ thống cần một phương pháp hợp nhất xếp hạng.

Reciprocal Rank Fusion (RRF) là một cách hợp nhất đơn giản và ổn định. Điểm RRF của một tài liệu d có thể được tính như sau:

$$Score_{RRF}(d) = \frac{1}{k + rank_{BM25}(d)} + \frac{1}{k + rank_{Vector}(d)} \quad (6)$$

Trong đó, $rank_{BM25}(d)$ và $rank_{Vector}(d)$ là vị trí của tài liệu trong từng danh sách kết quả, còn k là hằng số giảm ảnh hưởng của thứ hạng. RRF phù hợp khi kết hợp BM25 và tìm kiếm vector vì không yêu cầu hai loại điểm phải cùng thang đo.

0.4.3 Xếp hạng lại

Sau bước truy xuất ban đầu, hệ thống thường có một tập kết quả ứng viên. Reranking là bước đánh giá lại các ứng viên này với mô hình chính xác hơn. Trong khi bi-encoder mã hóa truy vấn và tài liệu riêng biệt để tìm kiếm nhanh, cross-encoder đưa cả cặp truy vấn và tài liệu vào cùng mô hình để đánh giá mức liên quan trực tiếp.

Cách làm này tốn chi phí tính toán hơn, nhưng chỉ áp dụng trên số lượng ứng viên nhỏ nên vẫn phù hợp trong pipeline RAG. Trong dự án, reranker được dùng để lọc nhiễu và chọn các đoạn ngữ cảnh cuối cùng trước khi đưa vào prompt.

0.4.4 Tối ưu hóa truy vấn

Truy vấn của người dùng thường ngắn, phụ thuộc ngữ cảnh hội thoại hoặc dùng cách diễn đạt khác với sách giáo khoa. Vì vậy, hệ thống có thể viết lại truy vấn hoặc sinh thêm truy vấn phụ để tăng độ phủ truy xuất. Query rewriting trong bối cảnh này không nhằm trả lời người dùng trực tiếp, mà nhằm tạo các biểu thức tìm kiếm rõ hơn cho RAG.

Khi câu hỏi phụ thuộc vào lịch sử hội thoại, hệ thống cần kết hợp chủ đề trước đó, lớp học hoặc bộ sách đã chọn vào truy vấn truy xuất. Điều này giúp các câu hỏi nối tiếp vẫn có đủ ngữ cảnh để tìm đúng tài liệu.

0.5 Adaptive Retrieval

Một chiến lược truy xuất cố định không phù hợp với mọi loại câu hỏi. Câu hỏi yêu cầu giải thích một khái niệm cụ thể cần các đoạn nội dung chi tiết, trong khi câu hỏi về chương trình học hoặc danh sách bài học lại cần tổng hợp metadata. Vì

vậy, hệ thống sử dụng cơ chế adaptive retrieval để chọn cách truy xuất theo loại truy vấn.

Trong dự án, bộ phân loại truy vấn được cài đặt bằng luật heuristic. Thành phần này không dùng LLM, mà dựa trên tín hiệu từ câu hỏi, lớp, chủ đề, bộ sách và kết quả phân tích intent. Cách tiếp cận này giúp chiến lược truy xuất dễ kiểm soát.

Chiến lược STANDARD được dùng cho truy vấn cụ thể nhưng thiếu phạm vi lớp hoặc chủ đề rõ ràng. Chiến lược BROAD phù hợp với câu hỏi tổng quan, khi hệ thống cần lấy các đoạn đại diện cho nhiều bài học. Chiến lược CURRICULUM ưu tiên cấu trúc chương trình, danh sách bài hoặc chủ đề, nên sử dụng metadata nhiều hơn là tìm kiếm vector chi tiết. Chiến lược HIERARCHICAL được dùng khi truy vấn có đủ thông tin về lớp hoặc chủ đề; hệ thống trước hết chọn bài học liên quan, sau đó tìm các đoạn con chi tiết trong phạm vi đã chọn.

Cơ chế này giúp hệ thống cân bằng giữa độ chính xác và độ bao phủ của truy xuất. Với các tác vụ dài như tạo slide hoặc giáo án, hệ thống có thể lấy nhiều chunk hơn so với hỏi đáp thông thường.

0.6 Điều phối yêu cầu và quản lý ngữ cảnh

Một trợ lý giáo dục không chỉ thực hiện một thao tác hỏi đáp đơn lẻ. Người dùng có thể yêu cầu tạo câu hỏi, chấm đáp án, giải thích câu sai, tạo slide hoặc tiếp tục một nội dung đã trao đổi trước đó. Vì vậy, hệ thống cần một lớp điều phối để phân tích ý định, lập kế hoạch hành động và duy trì trạng thái.

0.6.1 Phân tích ý định và lập kế hoạch hành động

Intent detection là quá trình xác định mục đích của người dùng từ câu nhập tự nhiên. Với hệ thống này, một câu có thể chứa nhiều ý định. Ví dụ, người dùng có thể yêu cầu vừa tạo slide vừa thêm câu hỏi luyện tập. Khi đó, hệ thống cần phân tích thành nhiều intent và lập nhiều hành động tương ứng.

Sau khi phát hiện intent, action planning ánh xạ ý định sang thao tác cụ thể của hệ thống. Cách tách intent routing và action planning giúp phân suy luận ngôn ngữ được tách khỏi phần quyết định nghiệp vụ. LLM có thể hỗ trợ nhận diện ý định, nhưng việc chọn service xử lý cuối cùng cần được kiểm soát bằng logic rõ ràng để tránh gọi sai chức năng.

0.6.2 Quản lý session và state

Session memory lưu lịch sử hội thoại và trạng thái học tập của người dùng. Trạng thái này cần thiết cho các yêu cầu nối tiếp như chấm câu trả lời, ôn lại câu sai hoặc duyệt dần ý slide.

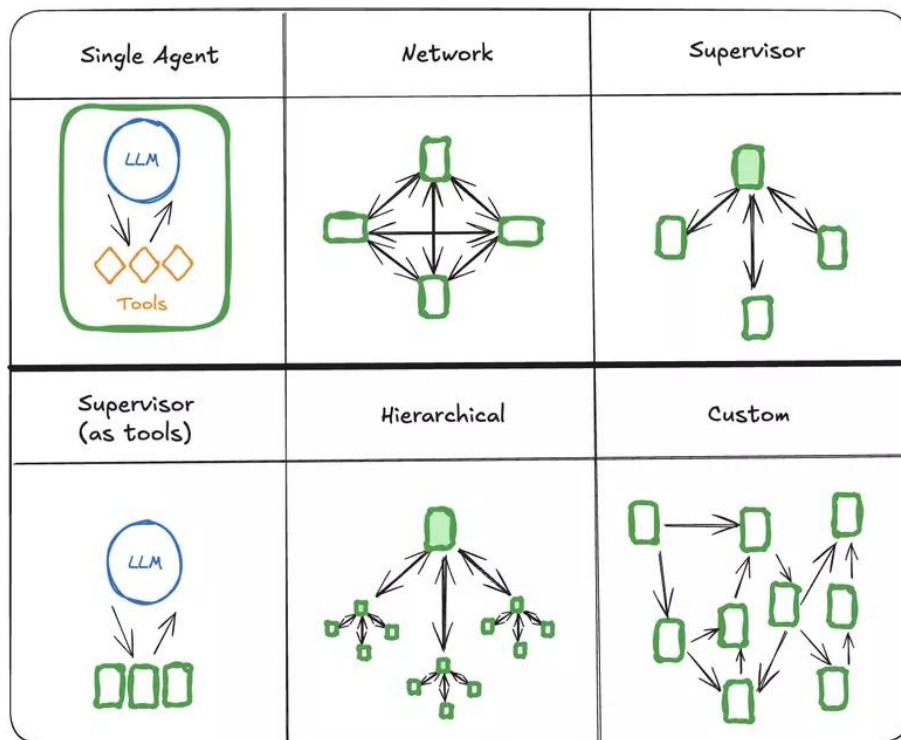
Trong kiến trúc hiện tại, request được biểu diễn bằng một đối tượng ngữ cảnh đi qua các bước xử lý. Đối tượng này lưu câu hỏi gốc, truy vấn đã làm giàu, intent, action plan, phạm vi sách/lớp, thông tin session và các bước debug. Cách tổ chức này giúp tránh truyền nhiều biến rời rạc giữa các module và hỗ trợ trace khi cần kiểm tra hệ thống.

0.7 Hệ thống đa tác tử và LangGraph

Multi-agent system là cách tổ chức trong đó nhiều tác tử chuyên biệt cùng tham gia giải quyết một tác vụ phức tạp. Mỗi tác tử đảm nhận một vai trò hẹp hơn, chẳng hạn lập dàn ý, viết nội dung, gợi ý media, tạo đánh giá hoặc kiểm tra chất lượng. Cách chia này phù hợp với các tác vụ dài vì đầu ra trung gian có thể được kiểm soát và kết hợp lại.

0.7.1 Tác tử AI và mô hình supervisor

Một tác tử AI thường gồm mô hình ngôn ngữ, chỉ thị nhiệm vụ và khả năng gọi công cụ. Trong kiến trúc supervisor, một tác tử điều phối quyết định tác tử chuyên môn nào cần được gọi ở từng bước. Mô hình này giúp kiểm soát luồng xử lý tốt hơn so với việc để các tác tử giao tiếp tự do.



Hình 0.1: Tổng quan các mô hình tổ chức trong hệ thống đa tác tử

Như minh họa ở Hình 0.1, hệ thống đa tác tử có thể được tổ chức theo dạng mạng lưới, hội đồng, chuỗi tuần tự hoặc supervisor. Đối với bài toán sinh slide và

giáo án, mô hình supervisor phù hợp vì các bước xử lý có thứ tự tương đối rõ: lập dàn ý, phát triển nội dung, bổ sung media, tạo hoạt động đánh giá, ghép kết quả và kiểm tra chất lượng.

0.7.2 Phạm vi áp dụng trong hệ thống

Trong đồ án này, multi-agent được áp dụng rõ nhất cho pipeline sinh slide và giáo án. Toàn bộ hệ thống không phải là một hệ LangGraph multi-agent end-to-end. Ở cấp cao, request vẫn được điều phối bởi Orchestrator, ActionPlanner và ExecutionDispatcher. LangGraph chỉ điều phối content pipeline, nơi tác vụ đủ dài và có nhiều artifact trung gian cần phối hợp.

Ranh giới này quan trọng vì các chức năng như quiz standalone có trạng thái tương tác riêng, bao gồm lượt làm bài, câu trả lời của học sinh, điểm số, câu sai và thống kê tiến độ. Nếu đưa toàn bộ quiz standalone vào content multi-agent pipeline, state học tập sẽ bị trộn với state sinh artifact và khó kiểm soát hơn.

0.7.3 A2A-lite và Human-in-the-loop

Để các tác tử trao đổi có cấu trúc, hệ thống sử dụng một contract nội bộ dạng A2A-lite. Contract này không phải là một runtime phân tán đầy đủ, mà là cách chuẩn hóa thông điệp trong cùng process. Một task gửi đến tác tử gồm mục tiêu, đầu vào, ràng buộc và artifact mong đợi; kết quả trả về gồm trạng thái, artifact, cảnh báo và metadata thực thi.

Human-in-the-loop được dùng trong bước duyệt dàn ý slide hoặc giáo án. Sau khi dàn ý được tạo, pipeline có thể tạm dừng để người dùng duyệt hoặc yêu cầu chỉnh sửa. Cơ chế này giúp kiểm soát cấu trúc nội dung trước khi hệ thống sinh phần chi tiết.

0.8 Tool layer và MCP-like protocol

Tool layer cho phép hệ thống chuẩn hóa cách các thành phần gọi công cụ bên ngoài hoặc công cụ nội bộ. Một công cụ cần có tên, mô tả, schema đầu vào và đầu ra. Khi tác tử hoặc service cần sử dụng công cụ, yêu cầu được đóng gói thành request có cấu trúc rồi dispatch đến implementation tương ứng.

Trong dự án, MCP-like protocol hiện đóng vai trò chuẩn hóa tool call nội bộ và làm điểm mở rộng. Các tool như truy xuất tri thức hoặc định dạng context có thể được bọc dưới cùng một interface. Tuy nhiên, đây chưa phải là triển khai MCP phân tán ở môi trường vận hành thực tế. Những năng lực như tìm kiếm media, kiểm tra URL hoặc gán nguồn tham khảo là hướng phù hợp để mở rộng tool layer trong tương lai.

0.9 Đánh giá chất lượng hệ thống RAG

Đánh giá hệ thống RAG cần xem xét cả chất lượng truy xuất và chất lượng sinh câu trả lời. Nếu tài liệu truy xuất không đúng, câu trả lời có thể sai hoặc thiếu. Nếu tài liệu đúng nhưng mô hình sinh không bám sát tài liệu, hệ thống vẫn có thể tạo ra hallucination.

0.9.1 LLM-as-a-Judge

LLM-as-a-Judge là cách dùng một mô hình ngôn ngữ làm bộ đánh giá tự động. Mô hình đánh giá có thể so sánh câu trả lời với ngữ cảnh, câu hỏi và đáp án tham chiếu để cho điểm theo một tiêu chí cụ thể. Cách tiếp cận này giúp kiểm thử trên nhiều mẫu nhanh hơn so với đánh giá thủ công hoàn toàn.

0.9.2 Các thước đo RAG

Faithfulness đo mức độ các luận điểm trong câu trả lời có được hỗ trợ bởi ngữ cảnh truy xuất hay không:

$$Faithfulness = \frac{|V \cap C|}{|V|} \quad (7)$$

Trong đó, V là tập luận điểm rút ra từ câu trả lời, còn C là tập luận điểm có thể suy ra từ ngữ cảnh.

Answer Relevance đo mức độ câu trả lời bám sát câu hỏi. Một cách tính phổ biến là sinh ngược các câu hỏi giả định từ câu trả lời, sau đó so sánh embedding của chúng với câu hỏi gốc:

$$Answer\ Relevance = \frac{1}{N} \sum_{i=1}^N \cos(E_q, E_{g_i}) \quad (8)$$

Trong đó, E_q là embedding của câu hỏi gốc, còn E_{g_i} là embedding của câu hỏi giả định thứ i .

Context Recall đo mức độ ngữ cảnh truy xuất bao phủ các thông tin cần thiết trong đáp án tham chiếu:

$$Context\ Recall = \frac{|GT \cap R|}{|GT|} \quad (9)$$

Trong đó, GT là tập thông tin thiết yếu từ đáp án tham chiếu, còn R là tập thông tin có trong ngữ cảnh truy xuất.

Context Precision đánh giá khả năng xếp hạng các tài liệu hữu ích lên vị trí cao:

$$Context\ Precision = \frac{\sum_{k=1}^K (Precision@k \times v_k)}{\text{Tổng số tài liệu liên quan trong } K} \quad (10)$$

Trong đó, K là số tài liệu được xét, $v_k = 1$ nếu tài liệu tại vị trí k có chứa thông tin liên quan và $v_k = 0$ nếu ngược lại.

Bên cạnh các thước đo RAG, hệ thống còn cần các bước kiểm tra chất lượng ở mức ứng dụng. Với quiz standalone, câu hỏi được kiểm tra ở cấp bộ câu hỏi và từng câu hỏi trước khi lưu vào session. Với slide và giáo án, kết quả sau khi ghép artifact được đưa qua bước quality review để quyết định chấp nhận, yêu cầu sửa hoặc dừng pipeline. Các cơ chế này bổ sung cho đánh giá định lượng và giúp giảm lỗi trong luồng sử dụng thực tế.

0.10 Kết chương

Chương này đã trình bày các nền tảng lý thuyết và công nghệ chính được sử dụng trong hệ thống, bao gồm biểu diễn ngôn ngữ, mô hình ngôn ngữ lớn, RAG, truy xuất lai, adaptive retrieval, điều phối intent, quản lý session, multi-agent, A2A-lite, HITL, tool layer và đánh giá RAG. Các nội dung này tạo cơ sở để hiểu vì sao hệ thống không chỉ gọi trực tiếp LLM, mà cần nhiều lớp kiểm soát để bảo đảm câu trả lời bám sát sách giáo khoa và phù hợp với từng tác vụ học tập.

Chương tiếp theo sẽ trình bày cách các nền tảng trên được hiện thực hóa trong kiến trúc hệ thống, bao gồm luồng xử lý request, các service chính, pipeline RAG, pipeline quiz, pipeline sinh slide/giáo án và kết quả đánh giá thực nghiệm.